

// HACKATHON 2026 SUBMISSION

AEGIS-AI

Technical Architecture & Object-Oriented Design

System architecture, OOP class design, pipeline flow, and the role of IBM Bob in AEGIS-AI's cyber-threat detection engine.

< 3s

Scan time

5

Risk tiers

100%

Private

IBM Bob

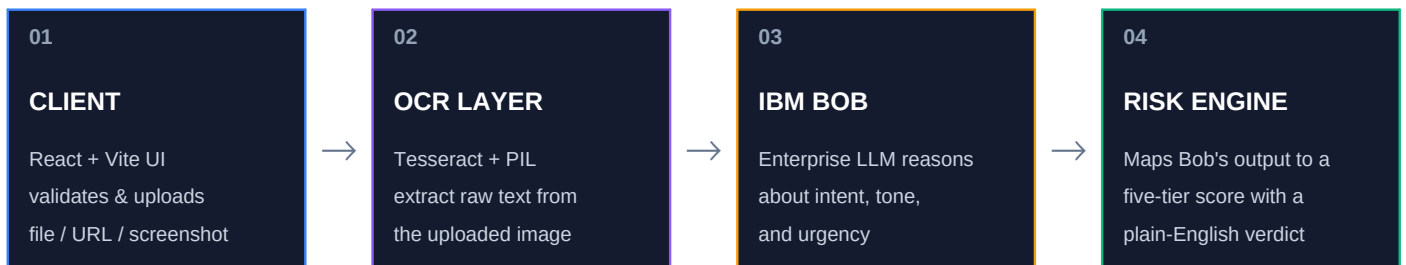
LLM engine

1. System Overview

AEGIS-AI bridges the gap between sophisticated cyber threats and everyday users who lack the tools to identify them. A person drops a screenshot, PDF, or link into the app; the system extracts the text, reasons about it using an enterprise LLM (IBM Bob), and returns a plain-English risk verdict in under three seconds. This document describes the system's layered architecture, its object-oriented class design, and how IBM Bob fits into the pipeline. The class design below is a conceptual / proposed design meant to document the system in standard OOP terms - it mirrors the pipeline shown in the product deck and demo, not a line-by-line reflection of a specific codebase.

2. High-Level Architecture

Every scan moves through four layers, front to back. Each layer has a single responsibility, which keeps the system loosely coupled - the frontend never talks to IBM Bob directly, and the risk engine never talks to the LLM directly either.



Client to Server: the React app validates the input client-side and posts it over HTTPS. OCR to LLM: extracted text (never the raw image) is what reaches IBM Bob, minimizing what leaves the OCR boundary. Risk Engine: the only layer that produces the final tier and copy shown to the user.

3. About IBM Bob

IBM Bob is IBM's enterprise AI platform. Rather than running a single fixed model, it routes each task across a mix of frontier LLMs, open-source models, and IBM's own Granite model family, choosing a model per task based on accuracy, latency, and cost - all behind one governed, secured interface built for enterprise workloads (compliance, hybrid/air-gapped deployment, and usage governance are first-class concerns for Bob, not afterthoughts).

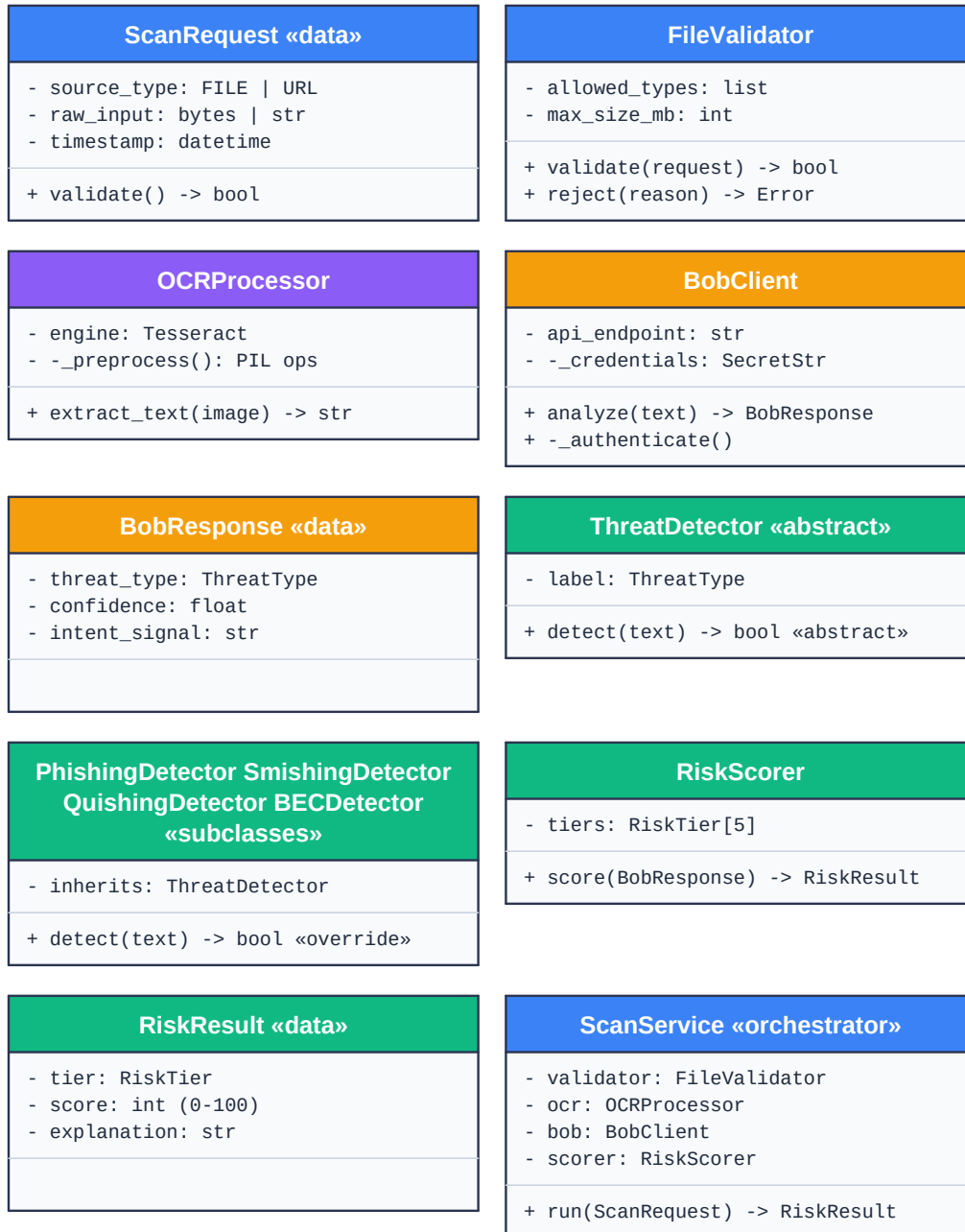
AEGIS-AI does not train or host its own classification model. It is a **client** of IBM Bob: once OCR has pulled the text out of an upload, AEGIS-AI's backend calls into Bob with that text and a classification instruction. Bob applies semantic reasoning - intent, tone, urgency - rather than keyword matching, and returns a structured result (threat type + confidence) that AEGIS-AI's Risk Engine turns into one of five color-coded tiers with a plain-English explanation.

Why this matters for the architecture:

- AEGIS-AI's own codebase stays small - it never re-implements NLP/LLM logic, it delegates to Bob.
- The `BobClient` class (see Section 4) is the single seam between AEGIS-AI and Bob - if the underlying model or IBM's API changes, only that one class needs to change.
- Because reasoning happens off-device on a governed enterprise platform, AEGIS-AI can make an explainability guarantee (every verdict ships with a reason) without building its own explainability layer.

4. Classes & Objects

The pipeline in Section 2 maps cleanly onto a small set of collaborating classes. Each class has one job; the **ScanService** orchestrates them in sequence. Attributes are marked with -, methods with +, following standard UML class-diagram notation.



Class names ending in «data» hold state only (no business logic) - they exist to pass a structured result between layers. «abstract» marks a class that is never instantiated directly; only its subclasses are.

5. Core OOP Principles Applied

AEGIS-AI's design leans on all four pillars of object-oriented programming. Each is grounded in a concrete class from Section 4 rather than left abstract.

Principle	Definition	Where it shows up in AEGIS-AI
Encapsulation	Bundling data and the methods that operate on it, and hiding internal state from the outside world.	OCRProcessor hides its Tesseract/PIL preprocessing behind a single <code>extract_text()</code> method - callers never touch pixel data directly.
Abstraction	Exposing only what a caller needs, and hiding implementation detail behind a simple interface.	BobClient exposes one method, <code>analyze(text)</code> , and hides HTTPS calls, auth tokens, and IBM Bob's internal model routing entirely.
Inheritance	A subclass reuses and extends the state/behavior of a parent class.	PhishingDetector, SmishingDetector, QuishingDetector, and BECDetector all inherit the <code>label</code> attribute and <code>detect()</code> contract from the abstract <code>ThreatDetector</code> base class.
Polymorphism	Different classes respond to the same method call in their own way, so calling code doesn't need to know which concrete class it's talking to.	ScanService (and, conceptually, RiskScorer) can call <code>detector.detect(text)</code> on any <code>ThreatDetector</code> subclass interchangeably - new threat types plug in without changing the orchestration code.

// SEQUENCE

6. Request Sequence (Step by Step)

- 1 User drops a file, screenshot, PDF, or URL into the React client.
- 2 FileValidator checks type and size client-side and again on the server.
- 3 OCRProcessor.extract_text() pulls every word out of the image, pixel by pixel (Tesseract + PIL).
- 4 ScanService builds a text payload and calls BobClient.analyze(text).
- 5 BobClient sends an authenticated HTTPS request to IBM Bob: POST /bob/analyze.
- 6 IBM Bob reasons about intent, tone, and urgency and returns a BobResponse (threat_type + confidence) as JSON.
- 7 RiskScorer.score() maps the BobResponse onto one of five tiers - Safe, Low, Medium, High, Critical.
- 8 ScanService returns a RiskResult to the client; the UI renders the tier, score, and plain-English explanation. No file or result is persisted server-side.

// RISK MODEL

7. Five-Tier Risk Model

Tier	Score	Meaning
Safe	0-15	No threat indicators found
Low	16-35	Minor caution advised
Medium	36-60	Verify before acting
High	61-85	Strong threat signals
Critical	86-100	Do not interact - report it

// THREAT COVERAGE

8. Threat Library

Threat Type	Category	Severity
Phishing	Social Engineering	High
Business Email Compromise	Fraud	High
Smishing (SMS Phishing)	Social Engineering	High
QR Code Phishing (Quishing)	Social Engineering	Medium
Safe / Benign Content	No Threat	Safe

9. Technology Stack

Layer	Technology	Responsibility
LLM Engine	IBM Bob	Enterprise reasoning / threat classification
OCR	Tesseract / PIL	Pixel-to-text extraction from any image format
Backend	Python / Flask	Lightweight REST API, no data persistence
Frontend	React + Vite	Fast, accessibility-first UI
Animations	Framer Motion	Smooth, motion-safe scroll animations

10. Design Principles

- Privacy by Default - files are processed in-memory and immediately discarded; nothing is stored, logged, or shared.
- IBM Bob-Powered - reasoning is backed by IBM's enterprise LLM platform, not a keyword classifier.
- Explainable Results - every RiskResult carries a plain-language explanation; no black-box output.
- Built for Everyone - no security knowledge required; results read in plain English with clear next steps.

Disclaimer: AEGIS-AI is an educational and awareness tool. Results are AI-generated guidance, not definitive legal or forensic analysis. The class design in this document is a conceptual OOP architecture prepared to document AEGIS-AI's pipeline in standard software-design terms; it is intended as a design reference, not a reverse-engineered listing of a specific codebase.